

Ex.No.13 :

Program :

```
#include <stdio.h>
```

```
// Best Fit Logic (used internally)
```

```
void bestfit(int mp[], int p[], int m, int n) {
```

```
    int j = 0;
```

```
    for (int i = 0; i < n; i++) {
```

```
        if (mp[i] >= p[j]) {
```

```
            printf("\n%d fits in %d", p[j], mp[i]);
```

```
            mp[i] = mp[i] - p[j++];
```

```
            i = -1; // restart scanning
```

```
        }
```

```
    }
```

```
    for (int i = j; i < m; i++) {
```

```
        printf("\n%d must wait for its process", p[i]);
```

```
    }
```

```
}
```

```
// Sort in descending order
```

```
void rsort(int a[], int n) {
```

```
    for (int i = 0; i < n; i++) {
```

```
        for (int j = 0; j < n; j++) {
```

```
            if (a[i] > a[j]) {
```

```
                int t = a[i];
```

```
                a[i] = a[j];
```

```
                a[j] = t;
```

```
            }
```

```
        }
```

```
    }  
}
```

```
// Sort in ascending order
```

```
void sort(int a[], int n) {  
    for (int i = 0; i < n; i++) {  
        for (int j = 0; j < n; j++) {  
            if (a[i] < a[j]) {  
                int t = a[i];  
                a[i] = a[j];  
                a[j] = t;  
            }  
        }  
    }  
}
```

```
// First Fit
```

```
void firstfit(int mp[], int p[], int m, int n) {  
    sort(mp, n); // ascending  
    sort(p, m);  
    bestfit(mp, p, m, n);  
}
```

```
// Worst Fit
```

```
void worstfit(int mp[], int p[], int m, int n) {  
    rsort(mp, n); // descending  
    sort(p, m);  
    bestfit(mp, p, m, n);  
}
```

```
int main() {
```

```
int m, n, mp[20], p[20], ch;
```

```
printf("Number of memory partitions: ");
```

```
scanf("%d", &n);
```

```
printf("Number of processes: ");
```

```
scanf("%d", &m);
```

```
printf("Enter memory partitions:\n");
```

```
for (int i = 0; i < n; i++) {
```

```
    scanf("%d", &mp[i]);
```

```
}
```

```
printf("Enter process sizes:\n");
```

```
for (int i = 0; i < m; i++) {
```

```
    scanf("%d", &p[i]);
```

```
}
```

```
printf("\n1. First Fit\t2. Best Fit\t3. Worst Fit\n");
```

```
printf("Enter your choice: ");
```

```
scanf("%d", &ch);
```

```
switch (ch) {
```

```
    case 1:
```

```
        firstfit(mp, p, m, n);
```

```
        break;
```

```
    case 2:
```

```
        bestfit(mp, p, m, n);
```

```
        break;
```

```
case 3:
    worstfit(mp, p, m, n);
    break;

default:
    printf("Invalid choice\n");
}

return 0;
}
```

Output

```
Number of memory partition : 5
Number of process : 4
Enter the memory partitions :
150
220
500
350
700
Enter process size :
160
450
500
412
1. Firstfit 2. Bestfit 3. worstfit
Enter your choice : 1

160 fits in 220
450 fits in 500
500 fits in 700
412 must wait for its process

=== Code Execution Successful ===
```